RESQGRID

Python + FastAPI implementation specification for Claude

Project thesis
A failure-aware emergency coordination simulator that recommends ambulance and destination choices, monitors unreliable information, proposes recovery actions, and measures outcomes against baseline dispatch.

Fixed stack: Python 3.12+, FastAPI, Uvicorn, Pydantic v2, MongoDB, Google OR-Tools, OSRM adapter plus mock router, FastAPI WebSockets, React + Vite + TypeScript + Tailwind CSS + Leaflet. Custom deterministic Python simulation; SUMO optional later.

Safety boundary: synthetic data only. No real emergency dispatch, hospital, traffic signal, emergency number or public messaging integration. No autonomous diagnosis or real-world dispatch actions.

Innovation	Backend	Proof
Failure detection + recovery	**Python / FastAPI / OR-Tools**	Repeatable replay against baseline policies.

Version 2.0 • Python/FastAPI edition • 22 October 2026

1. Exact technology stack

Layer	Technology	Purpose
Language	**Python 3.12+**	Backend, simulation, optimization, tests.
API	**FastAPI + Uvicorn**	Typed REST endpoints, OpenAPI docs, async support and WebSockets.
Validation	**Pydantic v2**	Validate API inputs/outputs and settings.
Database	**MongoDB**	Incidents, vehicles, hospitals, decisions, risks, audit and simulation runs.
Mongo driver	**Choose a maintained compatible async driver**	Inspect versions before selecting PyMongo Async or Motor; do not install both by default.
Optimization	**Google OR-Tools**	Constrained assignment after deterministic baseline works.
Routing	**OSRM adapter + local mock router**	Road estimates plus offline deterministic fallback.
Frontend	**React + Vite + TypeScript + Tailwind + Leaflet**	Command center, map, incident detail, hospital console and replay lab.
Realtime	**FastAPI WebSockets**	Push updates; reconnect and resync from REST state.
Simulation	**Custom deterministic Python event engine**	Seeded incidents and failure injection; SUMO optional.
Testing	**pytest + httpx**	Unit, API, integration and replay tests.
Quality/local	**Ruff; optional mypy; venv; Docker Compose optional**	Lint/type checks; core demo works without external keys.

Backend is Python/FastAPI—not Express. Frontend TypeScript is appropriate for the browser UI, but dispatch and simulation logic must live in Python.

2. Product workflow

- Dispatcher creates an incident manually or from optional typed/voice demo input.
- Incident Truth Engine stores reported facts, unknowns, provenance and confidence. Duplicate reports are suggested, never silently merged.
- Dispatch engine filters eligible ambulances and ranks vehicle/destination pairs using travel time, capability compatibility, resource freshness, coverage impact and reassignment cost.
- Dispatcher reviews explanation and approves, rejects or overrides the recommendation.
- Track sent, delivered-if-known, acknowledged, accepted, en route, on scene, transporting, arrived and handover as distinct states.
- Risk monitor detects stale GPS, stale hospital data, road closure, missed acknowledgment, ETA drift or competing incident.
- Recovery planner compares options and requires human approval before changing assignment/destination.
- Replay Lab runs identical seeded scenarios against baseline, optimized and recovery-enabled policies and exports metrics.

3. Functional requirements by module

M1 — Incident Truth and Fusion

- Fields: id, created_at, status, reported_category, operator_priority, patient_count, location_candidates, access_details, facts, unknowns, assigned_vehicle_id and destination_id.
- Accept manual form and typed text; optional speech-to-text/LLM extraction is an adapter, never required.
- Store field-level source, source timestamp, confidence and human_confirmed flag. Preserve raw reports separately from extracted candidates.
- Suggest duplicates using time window, geographic distance and text similarity. Human-confirmed merge/split is reversible and audited.
- Flag conflicting facts and ask only questions likely to change response. Approximate locations include uncertainty/radius.

M2 — Last-100-Metre Access

- Represent buildings with multiple entrances, gate restrictions, accessibility, landmarks and temporary access reports.
- Route to selected entrance node, not only building centroid. Show source, confidence and verification time.
- If entrance is uncertain, present alternatives and request confirmation. Temporary reports expire unless reconfirmed.

M3 — Vehicle State and Eligibility

- Vehicle fields: id, capability_tags, status, location, location_at, assigned_incident_id, availability_estimate and event history.
- Statuses: available, assigned, accepted, en_route, on_scene, transporting, unavailable. Validate transitions.
- Exclude unavailable/incompatible units. Stale location shows last-known position and age, never a fake live pin.
- Simulate offline/reconnected vehicles; handle duplicate and out-of-order updates idempotently.

M4 — Dispatch Optimizer

- Implement nearest-eligible baseline first, then travel-time-aware assignment.
- Hard constraints: availability and configured capability eligibility. Soft costs: travel time, freshness risk, coverage loss, uncertainty and reassignment switch penalty.
- Rank vehicle/destination pairs and show score components, constraints, assumptions and reason codes.
- Estimate post-dispatch coverage; document the prototype approximation. Require meaningful improvement and dispatcher approval before reassignment.

M5 — Hospital Capability and Freshness

- Separate stable hospital capabilities from current resource/capacity state.
- Each resource stores available/unavailable/unknown, last_confirmed_at and source.
- Stale data becomes unverified/unknown, never assumed available. Hospital role can confirm/deny/mark unknown; audit every change.
- Rank destinations using travel estimate, capability compatibility, freshness, capacity confidence and constraints.

M6 — Route Reality and ETA

- Define RoutingProvider interface with OSRMProvider and deterministic MockRoutingProvider.
- Inject road closure, congestion multiplier, inaccessible entrance and temporary event zones.
- Recompute after relevant closures or meaningful ETA drift. Store estimate time, source and confidence/range.
- Green-corridor animation is simulation only; no real signal integration.

M7 — Acknowledgment Watchdog

- Track created, sent, delivered_if_known, acknowledged, accepted, rejected and timed_out distinctly.
- Configurable timeout raises an alert and shows simulated fallback/escalation.
- Never treat sent as received or accepted. No real SMS/radio/phone calls.

M8 — Risk Monitor and Recovery Planner

- Triggers: stale vehicle location, no acknowledgment, route closure, stale hospital resource, capacity/capability change, ETA drift, state conflict and competing incident.
- Risk fields: id, incident_id, type, severity, evidence, detected_at, affected_assumption, recommended_actions and status.
- Options: keep current plan, request confirmation, reroute, alternate destination, propose eligible unit, or escalate.
- Estimate effects on time, suitability, coverage and other incidents. Show alternatives and uncertainty.
- Human approval required for assignment/destination changes. Never silently cancel active assignments.
- Use hysteresis, minimum-benefit thresholds and alert deduplication to avoid plan oscillation; audit every decision.

M9 — Hospital Pre-alert and Handover

- Pre-alert includes incident ID, reported reason, confirmed observations, unknowns, vehicle, ETA range and timestamp.
- Distinguish caller reports from measured observations and retain source/time.
- Hospital role acknowledges. Track arrival, handover_started, handover_completed and missing-data checklist.
- Calculate simulated arrival-to-handover time; restrict hospital view to assigned cases.

M10 — Incident Passport and Audit

- One timeline across report, correction, duplicate resolution, dispatch, vehicle/hospital updates, alerts, approvals and handover.
- Record actor, timestamp, old/new values, source and reason for important changes.
- Export history JSON/CSV; use stable event_id and correlation_id.
- Distinguish raw input, extracted suggestion, operator-confirmed field and simulated value.

M11 — Emergency Replay Lab

- Seeded synthetic world: incidents, ambulances, hospitals, resources, roads and disruptions.
- Compare P0 baseline, P1 optimized, P2 optimized-plus-recovery on identical seeds and information constraints.
- Inject duplicate calls, stale hospital data, blocked entrance, route closure, no acknowledgment, stale GPS, offline unit and competing incident.
- Save event timeline, policy decisions, seed, config, policy version and metrics; export JSON/CSV.
- Same seed/config reproduces results. Label all data simulated.

M12 — Optional AI Call Intake

- Optional speech-to-text + LLM adapter; never required.
- Strict Pydantic output schema, field confidence and source evidence. Invalid output/conflicts go to human review.
- AI must not diagnose, autonomously dispatch, downgrade urgency or reassign vehicles.
- Provide typed-input and mock mode; no API key required for core demo.

4. FastAPI backend architecture

Suggested project layout

```
app/
  main.py
  api/routes/{health,incidents,vehicles,hospitals,dispatch,risks,simulation,events}.py
  core/{config,logging,security}.py
  schemas/{incident,vehicle,hospital,dispatch,risk,simulation}.py
  services/{incident_service,dispatch_service,hospital_service,routing_service,risk_service,recovery_service,audit_service}.py
  optimization/{baseline.py,scoring.py,ortools_solver.py}
  simulation/{engine.py,scenarios.py,fault_injection.py,metrics.py}
  db/{mongo.py,seed.py}
  websockets/manager.py
tests/{unit,integration,replay}/
frontend/ (React/Vite UI)
scripts/{seed_demo,reset_demo,run_benchmark}.py
.env.example, README.md, pyproject.toml, docker-compose.yml (optional)
```

Keep business logic out of route functions. Routes validate requests and call services; services enforce business rules and state transitions.

Suggested API endpoints

Method/path	Purpose	Contract notes
GET /health	Health/readiness	Safe dependency status; no secrets.
POST /api/incidents	Create incident	Validate fields; mark demo mode.
GET /api/incidents/{id}	Incident detail/timeline	Include provenance and state.
POST /api/incidents/{id}/reports	Attach report	Suggest duplicate; no auto-merge.
GET /api/vehicles	List vehicles	Expose last-update age.
PATCH /api/vehicles/{id}/state	Update simulated vehicle	Validate transitions and actor.
GET /api/hospitals	List hospitals	Show resource freshness/unknown.
PATCH /api/hospitals/{id}/resources	Confirm resource	Audit actor/source/time.
POST /api/dispatch/recommend	Rank candidate assignments	Does not commit assignment.
POST /api/dispatch/{id}/approve	Approve assignment	Authorized actor + reason; audit.
POST /api/risks/evaluate	Evaluate triggers	Deterministic testing.
POST /api/risks/{id}/recover	Recovery proposal/action	Approval before plan mutation.
POST /api/simulation/runs	Run scenario	Policy, seed, fault profile.
GET /api/simulation/runs/{id}	Read results	Metrics + reproducible config.
WS /ws/events	Realtime updates	Reconnect/resync; authenticate in deployment.

Event envelope example

```
{"event_id":"uuid","event_type":"vehicle.location_updated","occurred_at":"ISO-8601","entity_id":"AMB-03","correlation_id":"INC-1007","source":"simulator","payload":{"lat":0.0,"lon":0.0,"accuracy_m":80}}
```

5. Data model and decision logic

Entity	Core fields	Rule
Incident	id, status, reported_category, operator_priority, patient_count, location_candidates, fields, vehicle_id, destination_id	Reported and confirmed facts separate.
Vehicle	id, capability_tags, status, location, location_at, assigned_incident_id	Stale location explicit; transitions validated.
Hospital	id, location, capabilities, resources, last_confirmed_at, source	Capability differs from current availability.
DispatchDecision	policy_version, candidates, constraints, scores, recommendation, approved_by, rationale	Persist enough to reproduce decision.
Risk	incident_id, type, severity, evidence, detected_at, actions, status	Deduplicate alerts.
AuditEvent	event_id, entity, type, occurred_at, actor, payload, correlation_id	Append-only timeline.
SimulationRun	seed, config, policy, metrics, result reference	Reproducible outputs.

Configurable scoring model (not clinically validated)

```
score = wT × travel_time_cost + wF × freshness_risk + wV × coverage_loss + wU × uncertainty_cost + wS × switch_cost
```

Apply hard constraints before scoring. Unavailable vehicles and infeasible destinations are excluded or marked infeasible. Show visible weights, component values and reason codes.

Recovery algorithm

- Detect timestamped trigger and safely ignore duplicate/out-of-order events.
- Create a risk with evidence, severity and affected plan assumption.
- Recompute affected plan elements; compare current plan and feasible alternatives.
- Include “keep current plan” where switching is not clearly beneficial.
- Display benefits, trade-offs and uncertainty; require authorized approval.
- Record action and continue monitoring.

6. Existing projects to inspect and improve

These are components/references to investigate, not mandatory dependencies. Claude must verify actual source, license, maintenance and compatibility before reuse. If unsuitable, implement a clean module and document why.

Tool/project	Reuse for	ResQGrid improvement	Reference
OpenStreetMap	Map/road/place data	Add entrances, synthetic incidents and closure overlay; follow attribution/licensing.	openstreetmap.org
OSRM	Road routing/duration	Adapter, mock fallback, ETA freshness/range, closure simulation.	project-osrm.org/docs
Google OR-Tools	Constraint optimization	Assignment constraints and coverage-aware comparison against baseline.	developers.google.com/optimization
SUMO	Optional traffic simulation	Add later for congestion/closures; must not block MVP.	sumo.dlr.de/docs
DispatchPulse repo	Candidate simulation reference	Inspect actual code/license/tests; reuse suitable logic with attribution.	github.com/Vishwas-Das/DispatchPulse
Dispatch_DAA repo	Candidate assignment/routing reference	Review algorithms; add recovery, freshness and coverage comparison.	github.com/Arun-Sanjay/Dispatch_DAA
Leaflet	Interactive map	Show freshness, route state and entrance details.	leafletjs.com

Repository due diligence

- Inspect source, README, manifests, tests, git status and license before editing.
- Check current activity, dependency health, Python compatibility and security issues.
- Document whether each project is reused, adapted, referenced with attribution, or rejected.
- Keep external services optional; public endpoints are not production guarantees.
- Preserve working code; no wholesale rewrite without a concrete reason.

7. Frontend and user experience

Page	Required behavior
Command Center	Map, incidents, vehicle/hospital states, alerts and stale-data badges; click incident to open timeline.
Incident Detail	Reported/confirmed/unknown facts, reports, entrance candidates and provenance; audited corrections.
Dispatch Compare	Ranked units/destinations, ETA range, constraints, score components, alternatives, approve/reject/override.
Risk & Recovery	Evidence, severity, suggested options and trade-offs; no hidden reassignment.
Hospital Console	Resource freshness, incoming pre-alert and acknowledgment action.
Replay Lab	Seed, policy selector, fault toggles, timeline, metrics, charts and exports.
Audit/Export	Chronological events with actor/reason/old-new values; JSON/CSV export.

- Use React + Vite + TypeScript + Tailwind + Leaflet; keep decision logic in Python.
- Show loading, empty, offline, error and degraded states; no fake telemetry.
- Use accessible labels and icons as well as color. Label simulated vs external data prominently.
- Every primary button must cause a tested, observable state change or be disabled/clearly marked unimplemented.

8. Simulation and benchmark

Implement a custom deterministic Python simulator first. SUMO is optional only if it improves evaluation after the MVP works.

Policy	Definition
P0 — Baseline	Nearest eligible available vehicle and simple destination selection; document assumptions.
P1 — Optimized	Travel-time-aware assignment, capability/freshness checks and coverage consideration.
P2 — ResQGrid	P1 plus failure detection, recovery options, acknowledgment tracking, switch penalty and audit.

Metrics

- Median and 90th-percentile response time on identical incidents.
- Time to appropriate/verified destination and proportion meeting configured requirements.
- Time to detect failure and time to recovery decision.
- Late/missed acknowledgments and dispatch-to-acceptance duration.
- Simulated arrival-to-handover duration.
- Unnecessary reassignments and plan oscillations.
- Coverage remaining after dispatch, including underserved zones.
- ETA interval calibration: fraction of arrivals inside predicted range.
- Use identical seeds/configs across policies and repeat across seeds.
- Report distributions, trade-offs and cases where ResQGrid performs worse.
- Do not leak future events to the policy under test.
- Export run config, seed, policy version and metrics. Never claim lives saved from simulation.

9. Safety, privacy, reliability and security

- Recommendations are advisory; authorized humans remain responsible for real-world decisions.
- No autonomous diagnosis, treatment, urgency downgrade or silent reassignment.
- No real emergency network, hospital, signal control or public messaging integration.
- Unknown is not available. Stale information is not current; show location age.
- Use synthetic data only; no real patient data or recordings in repository/demo.
- Use role-based access, least privilege, validation, audit access, safe logging and HTTPS in deployment.
- Keep secrets in environment variables; provide .env.example only.
- Define demo-data retention/deletion; raw audio not retained by default.
- Handle duplicate/out-of-order events idempotently; show connection and last-sync state.
- External failures cause a labelled mock/degraded state, never invented live data.

10. Phased build plan and acceptance criteria

Phase	Build	Done when
0. Inspect	Inspect repo, stack, tests, README, git status, license and current features.	Assessment and plan; no destructive rewrite.
1. Vertical slice	FastAPI, MongoDB, synthetic incident, 20 vehicles, 10–15 hospitals, map and timeline.	Create incident → recommend/approve unit → acknowledge → simulate route → arrival/handover.
2. Optimizer	Baseline then optimized assignment, destination suitability, freshness and coverage.	Deterministic ranked candidates and score explanation.
3. Recovery	Stale hospital/GPS, route closure, missing acknowledgment, competing incident.	At least five failures yield auditable proposals requiring approval.
4. Replay Lab	Seeds, P0/P1/P2, timeline, metrics and export.	Repeatable runs and actual comparisons.
5. Polish	Hospital console, pre-alert, responsive UI, docs and tests.	Fresh setup works; core demo needs no API key; data labelled simulated.

Required tests

- Unit tests: eligibility, destination compatibility, freshness, score breakdown, switch penalty.
- State tests: dispatch and incident lifecycle.
- Integration: incident → assignment → pre-alert → arrival → handover.
- Failure tests: route closure, stale GPS/hospital record and missing acknowledgment.
- Replay determinism: same seed/config yields same events and metrics.
- API validation and authorization tests.

11. Master prompt for Claude

Attach this PDF and the existing repository to Claude, then use the following as the first instruction.

You are the lead software architect and implementation engineer for ResQGrid. Follow the attached Python/FastAPI specification.

FIXED STACK
- Backend: Python 3.12+, FastAPI, Uvicorn, Pydantic v2.
- Database: MongoDB with a maintained compatible Python driver. Inspect versions before choosing PyMongo Async vs Motor; do not install both unnecessarily.
- Optimization: deterministic Python baseline first; Google OR-Tools after baseline works.
- Routing: OSRM adapter plus deterministic local mock router.
- Realtime: FastAPI WebSockets with reconnect/resync.
- Frontend: React + Vite + TypeScript + Tailwind CSS + Leaflet.
- Simulation: custom deterministic Python event engine; SUMO optional.
- Tests: pytest + httpx; Ruff and type checking where practical.
Do not change backend to Node/Express. Dispatch and simulation logic must remain in Python.

RULES
1. Inspect repository files, README, manifests, versions, tests, git status and entry points first. Do not overwrite working code without concrete reason.
2. Report repo assessment: existing functionality, reusable code, missing pieces, license/dependency risks and phased plan. Then implement.
3. Verify source/license/maintenance of any project before reuse. Preserve attribution; never assume features based on repository name.
4. Build working flows, not screenshot-only UI. Every primary button must have observable behavior and tests. Label placeholders clearly.
5. Core demo works offline without paid keys using synthetic data and mock routing. OSRM, LLM and SUMO are optional.
6. Implement in order: incident vertical slice; baseline dispatch; optimized dispatch; freshness/acknowledgments; recovery proposals with human approval; replay lab; hospital console/polish.
7. LLM can extract fields only. It must never autonomously dispatch, diagnose, downgrade urgency or reassign.
8. No real emergency/hospital/signal/public-message integrations. No clinical efficacy or lives-saved claims.
9. Add validation, error/loading/offline states, audit logs, tests, seed/reset scripts, .env.example and setup docs. Keep secrets out of source.
10. Run tests/build/lint after each phase. Report exact files changed, commands and actual results. Never claim unrun tests passed.

PRINCIPLES
- Unknown is not available; show source, confidence and freshness.
- Separate reported, measured, AI-extracted and human-confirmed facts.
- Sent, delivered-if-known, acknowledged and accepted are distinct states.
- Human approval required for assignment/destination changes.
- Use switch penalty/hysteresis to avoid unnecessary reassignment.
- Explain eligibility, score components, constraints, alternatives and uncertainty.
- Replay deterministically and compare policies on identical seeds.

START NOW
Inspect the repository and report assessment before editing. Then implement Phase 1 as a complete vertical slice, run tests, and proceed if feasible. If architecture conflicts with this spec, explain the trade-off and make the smallest safe change.

12. Reference links

• **FastAPI** — https://fastapi.tiangolo.com/

• **Pydantic** — https://docs.pydantic.dev/

• **MongoDB Python docs** — https://www.mongodb.com/docs/languages/python/

• **Google OR-Tools** — https://developers.google.com/optimization

• **OSRM** — https://project-osrm.org/docs/

• **SUMO** — https://sumo.dlr.de/docs/

• **OpenStreetMap** — https://www.openstreetmap.org/

• **Leaflet** — https://leafletjs.com/

- **India ERSS public features** — https://112.gov.in/features
- **DispatchPulse candidate repo** — https://github.com/Vishwas-Das/DispatchPulse
- **Dispatch_DAA candidate repo** — https://github.com/Arun-Sanjay/Dispatch_DAA

This is a prototype specification, not an approved emergency-service deployment plan. Validate with operational, legal, privacy and clinical stakeholders before real-world use.